THE LIVING DREAM

A persistent, multiplayer virtual universe shaped by everyone who enters it.



PROJECT SOURCE OF TRUTH · VERSION 1.0 · 23 OCTOBER 2026

This document captures the current product vision, core experience, universe design, multiplayer behavior, persistence rules, technical direction, MVP boundaries, and future possibilities. It is the reference point for planning and implementation until deliberately revised.

> **“A universe that never forgets. A world that nobody owns alone.”**

Document status & how to use it

Status: Concept baseline. This is the agreed product direction as of version 1.0, not a claim that the system has already been implemented or load-tested.

Decision rule

When design or implementation questions arise, use this document to preserve the central idea: one shared, persistent, surreal universe where users can meet in real time and collectively change the world. Technical choices may evolve, but they should support that experience rather than dilute it.

Contents

1. Vision, purpose, and product principles
2. Core experience and user journey
3. The multiverse: planets, themes, and rules
4. World interactions, creation, and dream physics
5. Avatars, characters, and social presence
6. Multiplayer behavior and synchronization
7. Persistence, permissions, and world history
8. Technical architecture and technology choices
9. Data model and event flows
10. Performance, scale, reliability, and safety
11. MVP scope and phased roadmap
12. Acceptance criteria and test scenarios
13. Visual references and art direction
14. Risks, non-goals, and future possibilities

Working principle: start with one excellent shared location and prove the full loop—join, meet, create, persist, disconnect, and return—before expanding into a large multiverse.

1. Vision, purpose, and product principles

LUCID is an interactive, persistent, multiplayer virtual universe. People can enter from anywhere, explore surreal worlds, encounter other people, build and modify places, and leave changes that remain visible to future visitors. It combines the social presence of a shared online world, the freedom of a sandbox, the atmosphere of a dream, and the visual ambition of an interactive digital artwork.

The defining promise

A user's actions become part of a shared world. Other users can witness those actions in real time when they are nearby, and persistent creations remain after the creator leaves.

What LUCID is

- **A shared world:** users inhabit the same authoritative universe, not isolated personal copies.
- **A persistent world:** committed changes survive page refreshes, disconnects, and server restarts.
- **A multiplayer world:** people can see nearby avatars, move, gesture, communicate, and create together.
- **A multiverse:** multiple planets and locations can have radically different visual styles and environmental rules.
- **A dream sandbox:** gravity, scale, material, time, weather, and space can behave in impossible ways.
- **A social creative experience:** users are not limited to consuming content; they can shape the environment.

What LUCID is not

- Not merely a 3D landing page or a sequence of pre-rendered scenes.
- Not a single-player exploration demo with locally saved changes.
- Not initially a competitive game, economy, or quest-driven RPG.
- Not an unrestricted code-execution environment; creative freedom is expressed through safe world tools and defined interaction systems.
- Not an internet-scale platform in the first release. Start with a small, reliable multiplayer experience.

Product principles

Principle	Meaning in the product
Shared reality	Users in the same location see the same committed world state.
Persistent creation	Buildings, placed objects, environmental changes, and discoveries can survive sessions.
Dream logic	World rules can be authored per planet and can intentionally defy real-world physics.
Freedom with boundaries	Users can create expressive worlds while permissions, moderation, and resource limits protect the community.
Discoverability	The world rewards curiosity through characters, hidden places, artifacts, and environmental storytelling.
Performance is part of UX	The world should feel responsive, load progressively, and adapt to device capability.

2. Core experience and user journey

The primary interface is the world itself. Conventional UI exists to support navigation, building, social interaction, settings, and accessibility; it should not overwhelm the environment.

A typical first session

1. **Arrival:** the user enters through a short, atmospheric introduction and arrives at a welcoming hub planet called The Origin.
2. **Orientation:** a lightweight tutorial teaches movement, camera control, interaction, and how to open the world map.
3. **Exploration:** the user discovers portals, landmarks, objects, characters, and other players.
4. **Encounter:** nearby users become visible with their avatars, movement, and simple gestures.
5. **Creation:** the user places an object or changes an allowed environmental property.
6. **Shared reaction:** nearby users see the change and can interact with or build upon it, depending on permissions.
7. **Persistence:** the server commits the change. It is still present when the user refreshes or returns later.
8. **Return:** a world journal or map helps the user revisit discovered locations and remember meaningful encounters.

Primary interaction modes

Mode	Purpose	Examples
Explore	Move through and discover the universe.	Walk, run, jump, fly, portal travel, inspect landmarks.
Socialize	Be present with other people.	See avatars, wave, emote, chat, gather, explore together.
Create	Change the shared environment.	Place, move, rotate, recolor, group, or remove objects.
Transform	Trigger dreamlike world effects.	Change gravity, sky, lighting, time, weather, or local scale.
Discover	Reveal authored and community-created content.	Find artifacts, unlock a story fragment, meet a character.
Personalize	Express identity and preserve progress.	Customize avatar, save creations, bookmark locations.

UX rule

Mystery should come from the world, not from confusing controls. The experience needs clear interaction cues, a way to recover from disorientation, keyboard and touch alternatives, and a simple return-to-hub option.

3. The multiverse: planets, themes, and rules

The universe contains multiple planets. Each planet has its own art direction, materials, atmosphere, inhabitants, and optional mechanics. The multiverse is not a set of differently colored copies; each planet should feel like a distinct dream with a coherent visual identity.

Planet design catalogue (initial concepts)

Planet / realm	Visual identity	Possible rules / activities
The Origin	Cosmic hub, monumental arches, floating fragments, soft starlight.	Onboarding, portals, social gathering, world map.
Dreamscape	Floating islands, clouds, impossible architecture, pastel twilight.	Low gravity, drifting platforms, hidden pathways.
Candyverse	Candy-colored architecture, gummy hills, sugar-glass rivers, oversized sweets.	Bouncy surfaces, edible-looking but non-consumable props, playful physics.
Pixel Realm	Voxel landscapes, pixelated textures, deliberately low-resolution art.	Grid-based building, block placement, retro visual effects.
Realmscape / Nature Prime	Near-real forests, mountains, lakes, weather, natural lighting.	Day/night cycles, gardening, environmental restoration.
Robotics / Mech Planet	Metallic cities, mechanical fauna, modular machines, robotic inhabitants.	Mechanical transformations, moving platforms, machine assembly.
Glassia	Transparent crystal cities, refracted light, reflective surfaces.	Light puzzles, translucent construction, refractive portals.
The Abyss	Deep oceans, bioluminescent life, dark blue expanses, submerged ruins.	Underwater movement, pressure-free exploration, glowing trails.
Cloud City	Floating districts, airships, bridges, enormous cloud formations.	Flying, wind currents, floating architecture.
The Fracture	Surreal geometry, shifting corridors, distorted horizons.	Local gravity changes, looping spaces, reality transitions.
Flora	Bioluminescent plants, giant mushrooms, glowing pollen, living structures.	Plants react to proximity; areas evolve through shared events.
The Void	Sparse, dark, abstract space with distant celestial objects.	Quiet exploration, scale shifts, portals and user-made art.

Planet consistency

- Every planet has a stable identity: palette, material language, lighting, sound, scale, and characteristic silhouettes.
- Each planet can define a limited set of world rules. For example, gravity may be reduced in Dreamscape and block-aligned construction may be preferred in Pixel Realm.
- A planet's rules must be authoritative and consistent for all users in the same location.
- Worlds can contain user-created regions, but moderation and ownership rules still apply.
- Planets are loaded as manageable regions and assets; the client should not load the entire multiverse at once.

4. World interactions, creation, and dream physics

LUCID should feel open-ended. The first release will not support literally arbitrary actions; it will provide a flexible toolkit that can be expanded without redesigning the entire world engine.

Building and object manipulation

- Place objects from a curated catalogue of primitives, props, plants, lights, structures, and themed assets.
- Move, rotate, scale, recolor, duplicate, group, and delete objects.
- Use snap-to-grid or free placement depending on the planet and selected tool.
- Save reusable creations or blueprints where supported.
- Preview a change before committing it, with clear confirmation for destructive actions.
- Show creator attribution and permissions for community-created objects.

Dream mechanics

Mechanic	Shared-world behavior	Implementation note
Gravity zones	Objects and avatars in a zone follow that zone's defined gravity rule.	Start with a small set of approved gravity presets.
Portals	Users travel to a destination location through a shared portal.	Validate destination and load the new location before switching presence.
Environmental transformation	A trigger changes lighting, sky, weather, or world state for everyone nearby.	Represent as a versioned world-state change or durable event.
Reactive flora	Plants respond to proximity, interaction, or a shared milestone.	Use deterministic server-approved states rather than independent client guesses.
Interactive artifacts	Objects reveal stories, audio, effects, or new routes.	Keep content data-driven and permission-aware.
Custom triggers	Authorized creators connect safe trigger types to predefined actions.	Do not execute arbitrary user-provided server code.

Shared events

The world may occasionally host collective events: a portal opens, a constellation appears, a forest illuminates, or a city changes state after a group action. The server should determine whether the event occurs and publish a shared result so clients do not disagree about the world.

MVP boundary

Begin with object placement and a small number of deterministic effects. Advanced physics, unrestricted construction, scripting, destructible terrain, and complex user-generated mechanics belong to later phases.

5. Avatars, characters, and social presence

LUCID is a social world, not an empty simulation. The world should contain both human-controlled avatars and authored characters that give places personality.

Player avatars

- Users can choose a starting avatar and customize supported appearance options.
- The avatar has a stable identity across sessions, even when the user changes planets.
- The initial movement set includes idle, walk, run, jump, and a small set of expressive gestures.
- Other nearby users see approximate position, orientation, and animation state in real time.
- A disconnected user is removed from active presence after a timeout; their persistent creations remain.
- Avatar rendering should support graceful lower-detail modes for less capable devices.

Authored characters

Character concept	Role in the world
The Wanderer	A recurring traveler who appears in different planets and leaves clues.
The Archivist	Preserves fragments of stories, memories, and world history.
Echo	A mysterious entity whose appearance or behavior changes with the environment.
The Architect	Introduces building tools and demonstrates how the world can be shaped.

Character behavior in the first version

Use scripted dialogue, simple state machines, proximity triggers, and authored animations. AI-generated dialogue can be explored later, but it should not be required for the core world to function. Characters should be consistent for all users who encounter the same shared event or state.

Social features

- Nearby-user list and avatar name labels, with privacy options.
- Text chat scoped to nearby users or a location.
- Gestures and emotes such as wave, sit, dance, and point.
- Optional group or party feature for friends exploring together.
- Block, mute, report, and hide controls.
- Clear indicators when an object is private, shared, or publicly editable.

6. Multiplayer behavior and synchronization

Multiplayer is a foundational feature, not a later visual effect. Two separate browsers should be able to enter the same location, see each other's avatars, observe movement, and witness shared world edits.

Two distinct state categories

State type	Examples	Expected handling
Transient session state	Avatar position, rotation, current emote, typing/presence status.	Sent frequently; broadcast to relevant nearby users; generally not stored as permanent history.
Persistent world state	Placed object, edited structure, portal destination, environmental setting, unlocked public artifact.	Validated by server, durably saved, versioned where needed, then broadcast to subscribed users.

Join and synchronization flow

1. Client authenticates and requests entry to a location.
2. Server checks access rules and registers the user in that location.
3. Server provides a consistent snapshot of the location's current persistent state.
4. Client subscribes to live events and reconciles any updates that occurred during snapshot loading.
5. Server sends nearby-user presence and movement state.
6. Client renders avatars and world objects; movement is interpolated for smoothness.
7. When the user leaves, the server updates presence and informs relevant clients.

Movement updates

Movement is high-frequency, transient state. The client sends input or movement snapshots at a controlled rate; the server validates and relays relevant state. Other clients interpolate between updates rather than snapping avatars directly from one packet position to the next. A production implementation may add prediction and reconciliation if needed.

Persistent edit flow

1. Client submits an action such as create, move, or remove an object.
2. Server validates the user's identity, permissions, object limits, location, and payload.
3. Server commits the change to persistent storage or a durable change log.
4. Server acknowledges the committed version and broadcasts the change to subscribers.
5. Clients apply the event idempotently and reconcile against the authoritative version if necessary.

Location and interest management

Do not broadcast every movement update to every connected user in the universe. Subscribe users to their current location and, when necessary, nearby spatial cells. This reduces network traffic and prevents clients from rendering irrelevant avatars and objects.

Initial multiplayer target

Target a small room of roughly 10–20 simultaneous users for the first meaningful test. This is a planning target, not a performance guarantee; it must be validated with load tests on defined server and client hardware.

7. Persistence, permissions, and world history

The world should remember committed creations. Persistence is what makes LUCID different from a temporary multiplayer scene.

What persists

- User-created structures and placed objects.
- Object transforms, materials, supported settings, and ownership.
- Planet or location configuration changes made by authorized users.
- Unlocks, discoveries, bookmarks, and selected user preferences.
- Portals and links created by authorized users.
- Selected community events or artifacts intended to remain in the world.

What normally does not persist

- Every intermediate avatar movement packet.
- Temporary camera movement or UI state.
- Uncommitted previews in build mode.
- Transient visual effects unless the effect changes a durable world state.
- Typing indicators and short-lived presence messages.

Permissions model

Permission	Meaning
Private	Only the creator or explicitly invited collaborators can view or modify the creation, where the product supports private spaces.
Shared edit	A defined group can modify the object or structure.
Public view	Anyone can see the creation, but only authorized users can change it.
Public edit	Anyone meeting the location's rules can contribute; destructive actions may require extra confirmation or creator approval.
Protected landmark	A system-authored or moderated location cannot be freely overwritten by ordinary users.

Conflict handling

- Each persistent object has an ID and version.
- Updates include the expected version or equivalent concurrency token.
- If two users edit the same object concurrently, the server rejects stale writes or applies a documented merge rule.
- Delete operations are validated and can be soft-deleted or recoverable for a limited period.
- World events have unique IDs so clients can ignore duplicates safely.
- A snapshot plus event sequence/version allows a reconnecting client to restore current state without losing updates.

Reliability principle

Do not tell users a permanent change succeeded until the server has durably accepted it. A visible optimistic preview is acceptable, but it must be reconciled if the server rejects the action.

8. Technical architecture and technology choices

The architecture should be simple enough to build and debug as a small team, while preserving a path to scale. Start with a modular monolith rather than prematurely splitting the product into many services.

Logical architecture

Layer	Responsibilities
React client	World UI, controls, avatar rendering, scene loading, build tools, chat, discovery journal.
3D runtime	Three.js / React Three Fiber scene graph, animation, assets, camera, local visual effects.
HTTP API (Express)	Authentication, profiles, location metadata, saved creations, inventory, permissions, world-management operations.
Realtime gateway (Socket.IO)	Location rooms, presence, movement updates, chat events, committed world-change broadcasts.
World rules / services	Validate actions, enforce permissions, resolve location rules, apply object and environment state transitions.
MongoDB	Users, locations, persistent objects, object versions, discoveries, world metadata, durable records.
Optional Redis	Cross-process presence/room coordination, rate-limit counters, ephemeral state or pub/sub when deployment requires it.
Asset storage + CDN	GLB models, textures, audio, thumbnails, environment assets.

Recommended initial stack

Concern	Choice	Reason
Language	TypeScript	Shared types and safer application code.
Frontend build	Vite + React	Fast iteration for a highly interactive client-side world.
3D	Three.js + React Three Fiber + Drei	Flexible browser 3D with React integration.
State	Zustand	Small, explicit client state for UI and world tools.
Styling	Tailwind CSS + targeted CSS	Rapid interface work without forcing every world visual into utility classes.
Realtime	Socket.IO initially	Practical rooms, reconnects, and event communication for the MVP.
API	Node.js + Express	Familiar MERN backend with a clear HTTP surface.
Database	MongoDB	Flexible world-object and location documents, with indexes designed around queries.
Tests	Vitest + Playwright	Unit tests and real browser workflows.

Important implementation note

Do not put every frame of animation or particle update through React state. Use the rendering loop and appropriate imperative APIs for continuous motion. Keep React state for meaningful application state such as selected objects,

tools, inventory, chat, and discovery progress.

9. Data model and event flows

This is an initial conceptual model. Field names and collection structure can change during implementation, but the relationships and responsibilities should remain clear.

Core entities

Entity	Representative fields	Purpose
User	userId, displayName, avatarConfig, preferences, createdAt	Account identity and appearance.
Planet	planetId, name, theme, assetManifest, worldRules, accessPolicy	Planet identity and configuration.
Location	locationId, planetId, regionId, metadata, version, accessPolicy	A manageable shared area with its own persistent state.
WorldObject	objectId, locationId, creatorId, type, transform, appearance, permissions, version, updatedAt	A persistent placed object or structure component.
Portal	portalId, sourceLocationId, destinationLocationId, transform, permissions	A validated connection between locations.
Discovery	userId, discoveryId, targetId, discoveredAt	User-specific progress and discovered content.
WorldEvent	eventId, locationId, actorId, type, payload, sequence, createdAt	Durable or auditable record of important changes where needed.
SessionPresence	sessionId, userId, locationId, position, rotation, lastSeenAt	Ephemeral active-user state; not necessarily a permanent MongoDB record.

Indexes and query patterns to plan for

- WorldObject indexed by locationId and objectId for loading a location snapshot.
- WorldObject indexed by creatorId for a user's creations page.
- Location indexed by planetId and regionId for world navigation.
- Discovery indexed by userId and targetId to prevent duplicate progress records.
- WorldEvent indexed by locationId and sequence/time if an event log is used.
- Use pagination or chunked loading for large object collections; do not send an entire planet's full object set to every client.

Event naming examples

Event	Direction	Purpose
location:join	Client → server	Request to join a location.
location:snapshot	Server → client	Current authoritative persistent state.
presence:join / presence:leave	Server → clients	Nearby user enters or exits.
avatar:state	Client → server → relevant clients	Validated transient movement or animation state.
world:object:create	Client → server	Request to create an object.
world:object:committed	Server → clients	Confirmed persistent object change.

Event	Direction	Purpose
world:object:update / delete	Client → server, then server → clients	Validated persistent edits.
world:environment:changed	Server → clients	Shared change to a location's environment.
chat:message	Client → server → location	Nearby text communication with moderation and rate limits.

10. Performance, scale, reliability, and safety

LUCID is visually ambitious, so performance must be treated as a product feature. A beautiful world that stutters, overheats a laptop, or takes too long to load is not a successful experience.

Rendering and asset strategy

- Load only the current location and nearby relevant assets; preload likely next destinations opportunistically.
- Use compressed textures and optimized GLB assets; avoid unnecessarily high polygon counts.
- Use instancing for repeated geometry such as stars, foliage, crystals, and similar objects.
- Use level of detail and distance-based visibility for large scenes.
- Avoid excessive transparent objects and expensive real-time shadows.
- Pool or reuse frequently spawned effects where appropriate.
- Offer quality settings and a reduced-effects mode for lower-powered devices.
- Measure frame time, memory, draw calls, asset load time, and interaction responsiveness.

Network and server strategy

- Throttle and validate movement updates; do not accept arbitrary client-provided state as authoritative.
- Send updates only to users who need them, using location rooms and eventually spatial interest management.
- Batch or coalesce high-frequency transient updates where appropriate.
- Apply payload size limits, rate limits, schema validation, and authentication checks.
- Use idempotent event handling and versioned persistent objects.
- Introduce Redis or multiple realtime instances only when a measured deployment requirement justifies them.

Safety and community controls

- Provide block, mute, report, and personal visibility options.
- Protect private or creator-owned objects from unauthorized changes.
- Set limits on object count, object size, asset types, chat frequency, and other resource-consuming actions.
- Never execute arbitrary user-supplied code on the server.
- Moderate user-created text and uploaded assets before they can affect other users.
- Design recovery mechanisms for destructive edits, abuse, and compromised accounts.
- Make privacy and presence expectations clear to users.

Initial measurable targets

Area	Initial target / validation
Multiplayer room	Test 10–20 simultaneous users in one location; record actual server and client performance.
World persistence	A committed edit remains after refresh, reconnect, and backend restart.
Visual responsiveness	Aim for a stable 60 FPS on a defined reference machine where feasible; provide graceful lower-quality modes.
Movement	Measure end-to-end latency under realistic network conditions and document the observed results.

Area	Initial target / validation
Join flow	Load a location snapshot progressively; measure time to first controllable frame and time to full quality.
Recovery	Reconnect without duplicating world objects or losing committed changes.

11. MVP scope and phased roadmap

The project can grow into a large virtual universe, but the first release must prove the essential promise with a deliberately small scope.

Phase	Build	Exit criteria
0. Foundations	Repo setup, TypeScript, scene bootstrap, camera, input controls, basic asset pipeline.	One navigable environment runs reliably in a browser.
1. First world	One polished hub location, avatar movement, collision boundaries, 3–5 interactive objects.	A user can explore and interact without multiplayer.
2. Real-time presence	Authentication or guest identity, location rooms, avatar sync, interpolation, join/leave.	Two browsers see each other moving in the same place.
3. Persistent objects	Object placement, server validation, MongoDB persistence, committed-change broadcasts.	User A creates an object; User B sees it; both reconnect and still see it.
4. Social basics	Gestures, nearby chat, user list, mute/block/report basics.	Two users can meet, communicate, and leave safely.
5. Dream mechanics	One portal, one environmental transformation, one special planet rule.	A shared event affects all users in the location consistently.
6. Multiverse	Two or three distinct planets, portal travel, location snapshots, progressive assets.	Users can travel between worlds without losing persistent state.
7. Creator depth	Groups, reusable blueprints, advanced permissions, discovery journal.	Users can make larger creations and manage who can change them.
8. Scale and polish	Profiling, spatial interest management, recovery, deployment/load testing.	Measured capacity and reliability are documented; no unsupported scale claims.

MVP definition

The MVP is complete when two users can join the same location, see each other's avatars moving, create and edit a shared object, observe the change in real time, disconnect, and return later to find the committed object still present.

12. Acceptance criteria and test scenarios

These tests convert the product vision into observable behavior. They should be used as implementation gates rather than relying only on visual demos.

Core multiplayer tests

1. **Two-user join:** User A and User B enter the same location in separate browsers and can see each other's avatars.
2. **Movement:** User A walks and turns; User B sees smooth, reasonably current movement without severe snapping.
3. **Presence cleanup:** User A closes the browser; after the defined timeout, User B no longer sees User A as active.
4. **Different locations:** Users in separate locations do not receive irrelevant high-frequency movement updates.
5. **Reconnect:** User A reconnects and receives current location state and current nearby presence.

Persistence tests

1. **Create:** User A places an object; the server validates and stores it; User B sees it after the committed event.
2. **Refresh:** both clients refresh; the object is loaded from the authoritative snapshot.
3. **Restart:** the backend restarts; the object remains available.
4. **Concurrent edit:** two users update the same object; stale changes are handled by the documented version policy.
5. **Unauthorized edit:** a user without permission cannot modify or delete a protected object.
6. **Duplicate event:** replaying the same committed event does not create duplicate objects.

World and UX tests

1. Portal travel loads the destination and updates the user's active location.
2. A shared environmental transformation appears consistently for all users in the location.
3. The world remains navigable with keyboard controls and has usable touch controls on supported mobile devices.
4. Reduced-motion preferences disable or reduce nonessential camera movement and effects.
5. Loading, disconnected, failed-save, and reconnect states are clearly communicated.
6. Users can mute or block another user and report inappropriate behavior.

13. Visual references and art direction

The following generated concept boards are visual direction references, not final production assets or exact UI specifications. Use them to communicate atmosphere, planet variety, and the intended relationship between immersive world visuals and lightweight interface elements.



Reference board: LUCID identity, planet variety, social exploration, creative building, avatar customization, and environmental interaction. Some text inside the concept image is illustrative and should not be treated as a product requirement.

Art direction principles

- **Distinct planet identities:** each world should be recognizable from silhouette, material, palette, lighting, and motion—not just a color swap.
- **Cinematic but usable:** strong composition and atmosphere must not hide important controls or make navigation confusing.
- **Layered depth:** foreground, midground, distant landmarks, atmospheric effects, and moving details should create a sense of scale.
- **Readable avatars:** players should remain identifiable against visually rich environments.

- **Material storytelling:** candy, pixel, glass, robotic, natural, and cosmic worlds should use different rendering and interaction rules where practical.
- **World-first UI:** keep menus contextual and minimal; use clear panels for building, chat, maps, inventory, and settings.
- **Accessibility:** provide text alternatives, readable contrast, scalable UI, input alternatives, and reduced-motion support.

Suggested reference research

Study interactive digital art, stylized game environments, virtual social worlds, architectural visualization, generative art, and high-quality WebGL experiences. References should inform composition and interaction—not be copied directly. Maintain a small visual reference library per planet, including lighting, palette, silhouette, material, animation, and UI treatment.

14. Risks, non-goals, and future possibilities

Risks to manage early

Risk	Why it matters	Mitigation
Scope explosion	A full multiverse, avatar system, creator tools, social systems, and multiplayer engine are each large projects.	Build one location first; use explicit phase exit criteria.
Rendering cost	Rich 3D scenes can overwhelm laptops and mobile devices.	Profile early, optimize assets, use instancing/LOD, provide quality settings.
Network inconsistency	Users may see different world states if updates are lost or reordered.	Authoritative server, versioned objects, snapshots, idempotent events.
Griefing	Public persistence makes destructive actions consequential.	Ownership, permissions, protected landmarks, undo/recovery, reports.
Unbounded user content	Large or unsafe content can harm performance and community safety.	Approved assets, quotas, validation, moderation, resource limits.
Premature infrastructure	Microservices and complex distributed systems increase development burden.	Start modular monolith; split only after measured need.
Visual polish before core loop	A beautiful demo can conceal broken persistence or multiplayer.	Prioritize the two-user create-and-return acceptance test.

Explicit non-goals for the first release

- Internet-scale concurrency or thousands of users in one scene.
- Full physics simulation across every object.
- Arbitrary user-uploaded executable scripts.
- A complete economy, marketplace, or token system.
- Photorealistic graphics on every device.
- AI characters that can do anything without constraints.
- Unmoderated public voice chat or unrestricted user-generated uploads.

Future possibilities (not MVP commitments)

- User-created planets or private dream spaces.
- Collaborative building sessions and shareable blueprints.
- More advanced avatar animation and emotes.
- AI-assisted characters with bounded memory and safety rules.
- Dynamic world events influenced by collective activity.
- Planet-specific creation systems and programmable interactions.
- Audio spaces, music, ambient sound, and shared performances.
- Larger regions with spatial interest management and multi-instance realtime infrastructure.
- Accessibility-focused navigation options and alternative low-motion modes.

The product in one page

LUCID is a persistent, multiplayer, dreamlike multiverse. People enter the same shared universe, travel between distinct planets, encounter other users and characters, create and transform environments, and leave changes that remain after they disconnect. Each planet has a unique visual identity and may define its own rules for gravity, materials, movement, weather, scale, and interaction.

The three promises

Promise	Proof
We share the world.	Two users in the same location see one another moving and witness the same committed environmental changes.
The world remembers.	A created object remains after refresh, reconnect, and backend restart.
Dream logic is possible.	At least one planet rule or shared environmental event changes the experience in a way that defies ordinary reality.

First milestone to build

One beautiful location. Two users. Real-time avatars. One object they can both see. A committed change that survives a restart. Everything else grows from that foundation.

Versioning note: update this document when a product decision materially changes. Record the decision, reason, and date so the source of truth remains useful rather than silently drifting.